

```

#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    printf("Parent process started\n");
    printf("Parent PID : %d\n", getpid());
    printf("Parent PPID : %d\n\n", getppid());

    pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }

    else if (pid == 0) {
        // Child process
        printf("Child Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Child PPID : %d\n", getppid());
        printf("Child State: Running\n");

        sleep(10);

        printf("Child State: Terminating\n");
        exit(0);
    }

    else {
        // Parent process
        printf("\nParent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);
        printf("Parent State: Running\n");

        sleep(5);

        printf("Parent is waiting for child...\n");
        wait(NULL);

        printf("Child completed.\n");
        printf("Parent State: Terminating\n");
    }

    return 0;
}

```

Output:

```
[venkat@Venkats-MacBook-Air ~ % nano week-3.c
[venkat@Venkats-MacBook-Air ~ % gcc week-3.c
[venkat@Venkats-MacBook-Air ~ % ./a.out
Parent process started
Parent PID : 74835
Parent PPID : 74802

Parent Process
Parent PID : 74835
Child PID : 74836
Parent State: Running
Child Process
Child PID : 74836
Child PPID : 74835
Child State: Running
Parent is waiting for child...
Child State: Terminating
Child completed.
Parent State: Terminating
```